Politecnico di Torino

Department of Control and Computer Engineering (DAUIN)
Master Degree in Mechatronic Engineering
Robotics

Project report: Nonlinear MPC for Collision Avoidance and Control of UAVs With Dynamic Obstacles

Group n°9
Davide Alban - 344963
Davide De Bortoli - 344975
Leonardo Urbani - 344982

June 30, 2025

1 Introduction

The *robotics project* is an optional activity with the objective of reproducing the result of a scientific paper in this field. Our group developed its activity around the paper *"Nonlinear MPC for Collision Avoidance and Control of UAVs With Dynamic Obstacles"*, presented by Björn Lindqvist, Sina Sharif Mansouri, Ali-akbar Agha-mohammadi, and George Nikolakopoulos in 2020 [1]. This document aims to present the analysis of the addressed problem, the methodology and considerations made during implementation phase, the main results obtained from simulations and possible extensions of the work.

2 Project development

2.1 Presented Control Architecture

The paper presents a Non-linear Model Predictive Control (NMPC) architecture to allow UAVs avoid the collision with dynamic obstacles ("projectiles") that are intersecting the drone trajectory.

For simplicity, the UAV model is defined in a yaw-compensated global frame of reference. This leads to a 8-states non-linear system (*position, velocity, roll and pitch*) controlled through a vector of three components: *roll, pitch and total trust.* The attitude of the drone is simply described in Euler form using the rotation matrix $R(\phi(t),\, \theta(t))$. The resulting equations to define the six degree of freedom for the UAV are:

$$
\begin{aligned}
\dot{p}(t) &= v(t) \\
\dot{v}(t) &= R(\phi(t),\, \theta(t)) \begin{bmatrix} 0 \\ 0 \\ T \end{bmatrix} + \begin{bmatrix} 0 \\ 0 \\ -g \end{bmatrix} - \begin{bmatrix} A_x & 0 & 0 \\ 0 & A_y & 0 \\ 0 & 0 & A_z \end{bmatrix} v(t) \\
\dot{\phi}(t) &= \tfrac{1}{\tau_\phi}(K_\phi \phi_{ref}(t) - \phi(t)) \\
\dot{\theta}(t) &= \tfrac{1}{\tau_\theta}(K_\theta \theta_{ref}(t) - \theta(t))
\end{aligned}
$$

where A_x, A_y, A_z are damping coefficients, τ_ϕ, τ_θ are time constants and K_ϕ, K_θ gain terms.

The above model is discretized using the forward Euler method to obtain $x_{k+1} = \zeta(x_k, u_k)$ and the result is used as *prediction model* of the NMPC with a prediction horizon of $N = 40$ steps. By associating a cost to a configuration of states and inputs at current time and in the prediction, it is possible to find an optimal control input vector minimizing the resulting *cost function*, formulated as:

$$
J(x_k, u_k, u_{k-1|k}) = \sum_{j=0}^{N} ||x_{ref} - x_{k+j|k}||^2 Q_x + ||u_{ref} - u_{k+j|k}||^2 Q_u + ||u_{k+j|k} - u_{k+j-1|k}||^2 Q_{\Delta u}
$$

where $Q_x, Q_u, Q_{\Delta u}$ are positive definite weight matrices for the states, inputs and input rates respectively. The constraints formulation is achieved defining $[h]_+ = max\,\{0, h\}$ and choosing h as an expression that is positive inside of the constrained area and negative outside of it. In particular, the h term has been defined as 1 over the difference between the square of the "safety radius" and the square of the distance between the UAV and the obstacle. In addition to that, we multiplied it for a penalty term that depends on the

kind of the predicted obstacle trajectory and we ensured, adding a constant term, that this function doesn't go to infinity for limit cases. A spherical object has been choose as obstacle and the underlying constraints are considered in the full control horizon N.
Additional constraints are applied on the magnitude of the change in control inputs ϕ_{ref} and θ_{ref} in order to limit aggressive or oscillatory behaviours of the UAV.
To predict the obstacle position, a function retrieves the best fitting movement between static, linear and parabolic by comparating the last 5 obstacle position and velocity measurements with the three different hypothesis.

Looking at the optimization, the NMPC problem is solved by a non-convex solver (PANOC) associated with software OpEn in which it is applied a penalty method. This is a technique to transform a constrained optimization problem into an unconstrained one by penalizing constraint violations in the objective function, ensuring that solutions that violate constraints are penalized, driving towards a feasible solution.

2.2 Code Implementation and Simplifications

Initially, we implemented the NMPC algorithm in a Python script using OpEn, assuming an idealized drone behavior. We tuned the parameters and verified proper obstacle avoidance with different configurations.

Then, we moved to the Gazebo simulator to evaluate the NMPC with a more realistic drone model. We used a simplified quadcopter model available at `https://github.com/PIC4SeR/simple_quadcopter_gazebo` which allows the drone to be controlled by publishing linear velocity commands along the three axes to the */cmd_vel*. As a result, in our code, it was necessary to convert the computed torques and inclinations into linear velocity commands. To represent the obstacle, we created a simple spherical object, which we controlled by publishing linear velocity commands to the */cmd_vel_sphere*.

We chose to use ROS 2, the latest version of the Robot Operating System, and developed a single node to handle both drone and obstacle control. After defining the main parameters and utility functions, we implemented two subscribers and two publishers. The subscribers continuously retrieve the state information of the drone and the obstacle from the */odom* and */odom_sphere*, providing real-time data on positions and velocities. This represents a necessary simplification since in the original paper the authors used a real drone and a series of cameras to infer the object position and velocities.

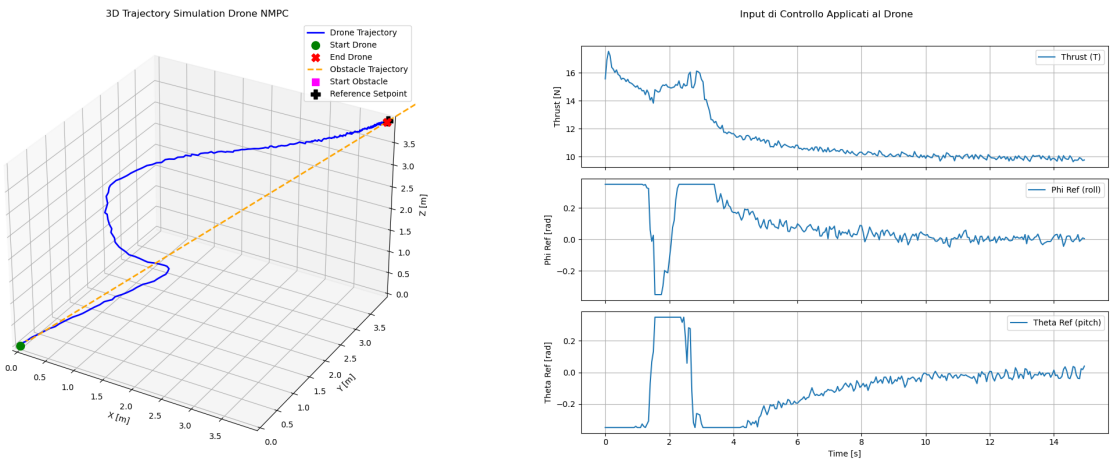
The trajectory of the obstacle is generated by publishing predefined linear velocities. For the drone, the NMPC computes appropriate linear velocities based on the current states of both the drone and the obstacle, enabling it to reach its target position while actively avoiding collisions.

2.3 Simulations

We tuned all the relevant parameters of the NMPC based on the behaviour observed in the Gazebo simulations, aiming to achieve a controller that is both effective and robust under realistic conditions. Numerous test scenarios were executed, varying both the goal position and the obstacle's behaviour, in order to comprehensively evaluate the system's performance.

The drone successfully avoids obstacles that intersect its ideal trajectory in different configurations, consistently generating smooth and collision-free paths. These results confirm the NMPC's capability to adapt its control strategy dynamically and maintain stability while navigating around moving obstacles.

However, limitations emerge when the obstacle moves at high velocities. In such cases, the drone may fail to react in time. This behavior highlights the importance of considering the drone's physical response limitations and latency.



(a) Drone and obstacle trajectories.

(b) Drone inputs.

Figure 1: Example of obstacle avoidance with linear moving obstacle.

3 Results and Possible implementations

Starting from the methodology proposed in "Nonlinear MPC for Collision Avoidance and Control of UAVs With Dynamic Obstacles", we developed a Nonlinear Model Predictive Controller (NMPC), solving the optimization problem in real time using OpEn.

The controller was tested in the Gazebo simulation environment to better approximate real-world conditions. Using ROS 2, we implemented a single node to manage both drone and obstacle control. NMPC parameters were tuned based on the behavior observed during simulations.

The results showed effective obstacle avoidance in various scenarios, with the drone navigating smoothly around moving obstacles. However, performance declined when the obstacle moved at high speed, limiting the drone's ability to react in time.

In future implementations, it would be valuable to incorporate onboard sensors for real-time obstacle detection. This would enable the system to identify dynamic obstacles based on actual perception data, taking into account measurement noise and other real-world uncertainties.

Moreover, a multiple-object avoidance could be easily implemented, probably requiring a re-tuning of the parameters to account for the new arising situations.